CHAPTER 18

Typed Ruby

When programming, when you see a variable in your code, it is useful to know what values can be assigned to that variable without the code breaking. For example, given the following Ruby method:

```
def mystery_method(x)
  x * 3
end
```

you would likely expect that `x` should be a number. But it's also completely valid Ruby for `x` to be a string (`"a" * 3` resolves to "aaa") or an array (`[:a] * 3` resolves to `[:a, :a, :a]`).

In the fullness of time, people call `mystery_method` with strings, integers, floating point numbers, and onward, until somebody changes the method and inadvertently changes what variables it will accept. For example:

```
def mystery_method(x)
  x.abs * 3
end
```

Now all of those string and array uses break because `abs` isn't defined for strings and arrays. If the original developer had been able to specify that `x` must be numeric, then the string and array uses would have found some other place to multiply by three and would not have broken when the method changed.

Historically, Ruby has gotten along just fine without requiring or allowing developers to augment code with this kind of information about the expected values of a variable, which is often called the *type* of the variable.

Spurred by the increasing complexity of large Ruby projects and the possibility of improved performance, Ruby 3.0 added a mechanism for allowing developers to specify type information about classes and methods. In addition, a third-party tool called Sorbet provides a separate mechanism for type control in Ruby. In this chapter, we take a look at both the official tool, called RBS, and at Sorbet.

What's a Type?

The terminology around types in programming languages can be confusing because each language community uses the terms slightly differently.